



SRI RAMACHANDRA

INSTITUTE OF HIGHER EDUCATION AND RESEARCH

(Category - I Deemed to be University) Porur, Chennai

SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

PYTHON PROGRAMMING LABORATORY

MED23CL202 · Semester III

LAB MANUAL

e-Manual — guided experiments with fill-in programs

Prepared by

Aishwarya S

Faculty in-charge

Programme

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND MEDICAL ENGINEERING

(Artificial Intelligence and Data Analytics)

ACADEMIC YEAR 2026 – 27



SRI RAMACHANDRA

INSTITUTE OF HIGHER EDUCATION AND RESEARCH

(Category - I Deemed to be University) Porur, Chennai

SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

PYTHON PROGRAMMING LABORATORY

MED23CL202 · Semester III

LAB RECORD

Submitted by

Name : _____

Register number : _____

In partial fulfilment of the requirements of the course, for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND MEDICAL ENGINEERING

(Artificial Intelligence and Data Analytics)

ACADEMIC YEAR 2026 – 27

Faculty in-charge : Aishwarya S

GUIDANCE — concept & hints

✓ How to use this notebook

This single notebook is your **lab manual + lab record** for the whole course. Each program has blanks written as `_____`. Read the **aim, algorithm** and the short guidance, then **replace every** `_____` with the correct code and run the cell with **Shift + Enter**. Compare your result with the *Expected output*.

Difficulty markers

Marker	Meaning
●	Basic — fill a single word/operator
●	Moderate — a line or a small block
●	Challenging — the mini-project

Blanks get harder as the course goes on — early experiments give you almost the exact answer in a comment; later ones give only the idea.

One project, ten steps: you also build a mini-project across the whole course — pick a track in the 🌱 *Mini-Project Tracks* cell before Experiment 1; after each experiment, a 🌱 guidance note gives your track's next step.

Before you submit / print: delete the *guidance* cells (click the cell, press `Ctrl+M` then `D`). What remains — Aim, Algorithm, your Program, its Output, Viva and Result — is your clean lab record. Export with **File → Print → Save as PDF**.

Viva tags: each viva question ends with its Bloom's level — (*K1 — Remember*), (*K2 — Understand*), (*K3 — Apply*), (*K4 — Analyse*) — so you know whether it tests recall, explanation, or hands-on transfer.

INDEX

Ex. no.	Date	Name of the experiment	CO	Page no.	Marks (10)	Staff sign
1		Data Types, Operators & Conditional Statements	CO1			
2		Loops, Collections & Functions	CO1			
3		Classes, Objects & Encapsulation	CO2			
4		Inheritance & Polymorphism	CO2			
5		Reading & Writing Text Files	CO3			
6		File Operations & CSV Records	CO3			
7		Exception Handling: try / except	CO4			
8		raise, finally & User-defined Exceptions	CO4			
9		Data Analysis with Pandas	CO5			
10		Visualization: Matplotlib & Seaborn	CO5			
—		Mini-Project — Final Assembly	CO1–CO5			

Ex. no.	Date	Name of the experiment	CO	Page no.	Marks (10)	Staff sign
—		Model Examination	CO1–CO5			

GUIDANCE — study map: learn it first in Foothold

Before each lab slot, read these units in the **Python Programming** Foothold course. Do that, and the experiment will feel like revision instead of new material.

Exp	Learn it first in Foothold
1	M4 Your First Python Program — Units 4.1–4.4 · M5 Making Decisions — Units 5.1–5.3
2	M6 Repetition — Units 6.1–6.3 · M7 Organizing Data — Units 7.2–7.4 · M8 Functions — Units 8.1–8.3
3	Units 10.1–10.3 — <i>Why Objects?</i> · <code>__init__</code> & <code>self</code> · <i>Methods & Encapsulation</i>
4	Unit 10.4 <i>Inheritance</i> · Unit 10.6 <i>Polymorphism & Abstraction</i>
5	Unit 9.3 <i>Files: Making Data Survive</i>
6	Units 9.3–9.4 · Unit 12.1 <i>Meet Pandas</i> (preview)
7	Units 9.1–9.2 — <i>Errors Aren't Failures</i> · <i>Catching & Raising</i>
8	Unit 9.2 <i>Catching & Raising</i> · Unit 10.7 <i>User-Defined Exceptions</i>
9	Unit 12.1 <i>Meet Pandas: DataFrames</i>
10	Units 13.1–13.3 <i>Matplotlib</i> · <i>Seaborn</i> · <i>Plotly</i>

This cell is guidance — delete it before printing your record.

GUIDANCE — Mini-Project Tracks: choose ONE in week 1

Instead of a separate capstone at the end, **your mini-project grows with you** — after every experiment, a  *grow your mini-project* note shows how to apply that week's concept to your project. Ten small steps later, you have a full, working application.

Pick ONE track now (or propose your own of similar scope to your faculty). You already know your destination — the table below is what your finished project will do by Experiment 10.

Track	
 Patient Vitals Monitor	Record patients' vitals from a menu, flag abnormal readings, keep the full
 Pharmacy Stock Manager	Track medicines with dispense/restock from a menu, warn on low stock via
 Clinic Appointment Book	Book, view and cancel OPD slots, refuse double-booking with a custom ex
 Health-Camp Screening Analyzer	Enter screening records (height, weight, BP), classify each visitor, store th

How it's assessed (at final submission): correctness 40% · code quality & comments 20% · breadth of course concepts used 20% · demo & viva 20%.

This cell and every  note are guidance — delete before printing your record. Your assembled project goes in the Mini-Project section at the end of this notebook.

✓ Experiment 1 — Data Types, Operators & Conditional Statements

CO: CO1 · Sessions: 1–2

Aim

To store and manipulate different **data types**, apply **arithmetic, relational and logical operators**, and take decisions using **conditional statements**, using simple patient-health data.

GUIDANCE — concept & hints

Concepts for this experiment

Data types hold different kinds of value: `int` (whole numbers, e.g. heart rate `72`), `float` (decimals, e.g. temperature `98.6`), `str` (text, e.g. a name), and `bool` (`True` / `False`). Check any value's type with `type(x)`.

Operators combine values: *arithmetic* `+` `-` `*` `/` `//` `%` `**` (note `**` is power, `//` is floor-division, `%` is remainder); *relational* `>` `<` `>=` `<=` `==` `!=` which give a `bool`; and *logical* `and`, `or`, `not` which combine conditions.

Conditionals choose which code runs: `if` tests a condition, `elif` adds more branches, `else` is the fallback. The 4-space **indent** marks the block that belongs to each branch.

Input: `input()` always returns a **string**, so wrap it in `int(...)` or `float(...)` before doing arithmetic.

✓ Program 1 — Arithmetic on Patient Vitals

Aim of program: Read a patient's weight (kg) and height (m) and compute the **BMI** using `BMI = weight / (height ** 2)`.

Algorithm

1. Read `weight` as a float.
2. Read `height` as a float.
3. Compute `bmi = weight / (height squared)`.
4. Print the BMI rounded to 2 decimals.

GUIDANCE — concept & hints

 **Fill ONE word per blank** — a function name, keyword or operator.

`float(...)` converts text to a decimal; `**` raises to a power; `round(x, 2)` keeps 2 decimals.

Expected output

```
Enter weight in kg: 60
Enter height in m: 1.6
Patient BMI = 23.44
```

```
# Program 1 : BMI calculator
weight = float(input("Enter weight in kg: "))
height = ____ (input("Enter height in m: "))      # convert the text to a float
bmi = weight / (height ____ 2)                  # ** raises height to the power
print("Patient BMI =", ____ (bmi, 2))            # function that rounds to 2
```

✓ Program 2 — Body-Temperature Unit Converter

Aim of program: Offer a menu — (1) Celsius→Fahrenheit, (2) Fahrenheit→Celsius — read the choice and a temperature, and convert it. $F = C * 9/5 + 32$, $C = (F - 32) * 5/9$.

Algorithm

1. Print the menu; read `choice` (int) and `temp` (float).
2. If choice is 1 → C→F; elif 2 → F→C; else → 'Invalid choice'.
3. Print the converted temperature.

GUIDANCE — concept & hints

 **Fill ONE word per blank** — the branch keywords/operator.

`==` tests equality; `elif` is the 'otherwise-if' branch; `else` is the catch-all.

Expected output

```
1. Celsius to Fahrenheit
2. Fahrenheit to Celsius
Enter choice: 1
Enter temperature: 37
Converted temperature = 98.6
```

```
# Program 2 : temperature converter using if / elif / else
print("1. Celsius to Fahrenheit")
print("2. Fahrenheit to Celsius")
choice = int(input("Enter choice: "))
temp = float(input("Enter temperature: "))

if choice == 1:                # relational operator: is choice equal
    print("Converted temperature =", temp * 9/5 + 32)
elif choice == 2:              # keyword for the 'otherwise-if' branch
    print("Converted temperature =", round((temp - 32) * 5/9, 2))
else:                          # keyword for the 'none of the above' branch
    print("Invalid choice")
```

✓ Viva-Voce Questions

1. Name the four built-in data types you used in this experiment, with one example each. (K1 — Remember)
2. `input()` always returns a string — explain what goes wrong in the BMI program if you forget `float()`. (K2 — Understand)
3. A patient enters weight 60 and height 0 — which line fails, and how would you guard it with a conditional? (K3 — Apply)

Result

Thus data types, operators and conditional statements were implemented in Python using patient-health data, and the outputs were verified.

GUIDANCE — grow your mini-project

Apply what you just practised to **your chosen track** (tracks are described before Experiment 1 — pick one there in week 1 and stay on it).

Track	This experiment's step
 A Vitals Monitor	Read one patient's temperature, pulse and systolic BP; use <code>if/elif/else</code> to print a status
 B Pharmacy Stock	Read one medicine's stock and its reorder level; print <code>REORDER</code> or <code>OK</code> using a conditional.
 C Appointment Book	Read an hour (0–23); print <code>OPEN</code> , <code>LUNCH BREAK</code> or <code>CLOSED</code> for your clinic's timings with
 D Camp Analyzer	Read one visitor's weight and height; compute BMI and print the WHO band with a condition

Next experiment: handle MANY patients/medicines/slots/visitors with loops, collections and functions.

✓ Experiment 2 — Loops, Collections & Functions

CO: CO1 · **Sessions:** 3–4

Aim

To repeat actions with `for` and `while` loops, store grouped data in **lists and dictionaries**, and organise reusable code with **functions** and **modules**, on patient data.

GUIDANCE — concept & hints

Concepts for this experiment

Loops repeat work. A `for` loop walks through a sequence — often `for i in range(n):` which counts `0,1,...,n-1`. A `while` loop repeats *as long as* a condition stays true. The **accumulator pattern** is the heart of most loops: start a variable at `0` before the loop (`total = 0`), then update it each pass (`total += x` to sum, or `count += 1` to count).

Collections hold many values in one variable. A **list** `[...]` is ordered and changeable (`append`, `index`, `slice`). A **tuple** `(...)` is ordered but *immutable*. A **set** `{...}` stores only *unique* items and supports `&` (common), `|` (all), `-` (difference). A **dict** `{key: value}` maps keys to values — look up with `d[key]`, add with `d[key] = value`, iterate pairs with `.items()`.

A **function** is a named, reusable block: `def name(params):` runs its body when *called*, and `return` hands a value back. A **module** is a ready-made library you `import` — e.g. `statistics` (`mean`, `median`) and `math` (`sqrt`, `pi`).

Exposure only (not practised here): **strings** are sequences too — index with `s[0]`, slice with `s[1:4]`, and use methods like `.strip()`, `.upper()`, `.replace(a, b)` and `.split(sep)`. Try them in a scratch cell.

✓ Program 1 — Average Heart Rate (for loop)

Aim of program: Read `n` heart-rate readings and print their average using a `for` loop.

Algorithm

1. Read how many readings `n`.
2. Set `total = 0` (the accumulator).
3. Loop `n` times, read each reading and add it to `total`.
4. Print `total / n` rounded to 2 decimals.

GUIDANCE — concept & hints

 **Fill ONE word per blank** — a built-in and an operator.

`range(n)` produces the loop counts; `+` adds each reading to the running total.

Expected output

```
How many readings? 3
Enter heart rate: 72
Enter heart rate: 80
Enter heart rate: 76
Average heart rate = 76.0
```

```
# Program 1 : average heart rate using a for loop
n = int(input("How many readings? "))
total = 0                                # accumulator starts at zero
for i in ____ (n):                        # built-in that produces 0,1,...,
    hr = int(input("Enter heart rate: "))
    total = total ____ hr                 # add this reading to the running
average = total / n
print("Average heart rate =", round(average, 2))
```

✓ Program 2 — Reusable BMI-Category Function

Aim of program: Write a function `bmi_category(weight, height)` that returns the WHO band, then call it for two patients.

Algorithm

1. Define `bmi_category(weight, height)`.
2. Inside, compute BMI and `return` the matching band.
3. Call it for two patients and print each returned value.

GUIDANCE — concept & hints

 **Fill the returned label and the second function call.**

Inside a function, `return "Normal"` hands the label back; call it like

`bmi_category(82, 1.7)`.

Expected output

```
Patient A: Normal
Patient B: Overweight/Obese
```

```
# Program 2 : reusable function that returns a BMI category
def bmi_category(weight, height):
    bmi = weight / (height ** 2)           # compute BMI from the two param
    if bmi < 18.5:
        return "Underweight"
    elif bmi < 25:
        return ____                        # label returned for the normal
    else:
        return "Overweight/Obese"

# call the function for two patients and print the returned value
print("Patient A:", bmi_category(60, 1.6))
print("Patient B:", ____ (82, 1.7))      # call the same function for pat
```

✓ Viva-Voce Questions

1. State, in one line each, when a `for` loop fits and when a `while` loop fits. (K1 — Remember)
2. In your BMI function, which names are parameters and which are arguments at the call site? (K2 — Understand)
3. A user answers `0` to “How many readings?” — trace exactly which line of the average program fails, and add the one check that fixes it. (K4 — Analyse)

Result

Thus loops, collections, functions and modules were implemented in Python on patient data, and the outputs were verified.

GUIDANCE — grow your mini-project

Apply what you just practised to **your chosen track** (tracks are described before Experiment 1 — pick one there in week 1 and stay on it).

Track	This experiment's step
 Vitals Monitor	Store several readings in a list; write <code>check_vitals(temp, pulse, bp)</code> returning a status
 Pharmacy Stock	Keep stock in a dict <code>{medicine: qty}</code> ; write <code>needs_reorder(qty, level)</code> ; loop over (
 Appointment Book	Keep the day in a dict <code>{slot: patient_or_None}</code> ; write <code>is_free(slot)</code> ; loop to print a
 Camp Analyzer	Keep records as a list of <code>(name, bmi)</code> ; write <code>bmi_band(bmi)</code> ; loop to count visitors per l

Next experiment: turn these loose variables and functions into a proper class.

✓ Experiment 3 — Classes, Objects & Encapsulation

Aim

To model real-world entities as **classes**, create **objects** with their own attributes and methods, and protect an object's data using **encapsulation** — private attributes changed only through validating methods — with medical examples.

GUIDANCE — concept & hints

Concepts for this experiment

A **class** is a blueprint that bundles **data (attributes)** with **behaviour (methods)**. The special method `__init__` is the *constructor*: it runs when you create an object and sets up its attributes. The first parameter `self` refers to the particular object, so `self.name = name` stores the value *on that object*.

An **object** (or *instance*) is one concrete thing built from the class: `p = Patient("Ravi", 45)` calls the class like a function. You then reach its data with `p.name` and run its behaviour with `p.show()`.

Encapsulation means keeping an object's data *and* the code that changes it together, while restricting direct outside access. A **private attribute** is named with two leading underscores (`self.__temp`); outside code cannot casually change it. Instead you provide **getters/setters** that control *how* the data may change — the natural place to add **validation** (e.g. reject an impossible temperature).

Exposure only: the `@property` decorator lets a computed value be read like an attribute (`p.bmi`, no parentheses), and **abstract base classes** (`abc` module) define *what* every subclass must provide without saying how.

✓ Program 1 — A Simple Patient Class

Aim of program: Define a `Patient` class with `name` and `age`, and a `show()` method that prints them; then create one object.

Algorithm

1. Define `Patient` with `__init__(self, name, age)`.
2. Store both values on `self`.
3. Add `show()` to print them.
4. Create a `Patient` object and call `show()`.

GUIDANCE — concept & hints

● Fill the attribute name and the class call.

`self.age = age` stores the age; `Patient("Ravi", 45)` builds an object.

Expected output

Name: Ravi | Age: 45

```
# Program 1 : a simple Patient class
class Patient:
    def __init__(self, name, age):
        self.name = name           # store the name on the object
        self.__ = age             # store the age on the object

    def show(self):
        print("Name:", self.name, "| Age:", self.age)

p1 = __("Ravi", 45)               # create a Patient object (call
p1.show()
```

✓ Program 2 — Validating Setter

Aim of program: Store a private `__temp` that a setter only accepts when it is a realistic human temperature (30–45 C).

Algorithm

1. Define `VitalRecord` with private `__temp = 0.0`.
2. `set_temp(value)` stores it only if $30 \leq \text{value} \leq 45$; else prints 'Rejected'.
3. `get_temp()` returns it.
4. Try a valid value, then an invalid one, then read it back.

■ GUIDANCE — concept & hints

● Fill an invalid test value and the getter call.

Any value outside 30–45 is rejected — e.g. `99`; read it back with `v.get_temp()`.

Expected output

Rejected: unrealistic temperature
Stored temperature: 37.5

```
# Program 2 : validate data using a setter
class VitalRecord:
    def __init__(self):
```

```

        self.__temp = 0.0                                # private temperature store

    def set_temp(self, value):
        # accept only realistic human temperatures
        if 30.0 <= value <= 45.0:
            self.__temp = value
        else:
            print("Rejected: unrealistic temperature")

    def get_temp(self):
        return self.__temp                                # controlled read access

v = VitalRecord()
v.set_temp(37.5)                                         # valid -> stored
v.set_temp(____)                                         # invalid (e.g. 99) -> rejecte
print("Stored temperature:", v.____())                  # call the getter

```

✓ Viva-Voce Questions

1. Define *class* and *object* in one sentence each. (*K1 — Remember*)
2. When `p1.show()` runs, what exactly does `self` refer to? And during `p2.show()`? (*K2 — Understand*)
3. Add a pulse attribute that must stay within 30–220 — where do you enforce it (constructor, setter, or both), and why? (*K3 — Apply*)

Result

Thus classes, objects and encapsulation were implemented in Python to model medical entities, and the outputs were verified.

■ GUIDANCE — 🌱 grow your mini-project

Apply what you just practised to **your chosen track** (tracks are described before Experiment 1 — pick one there in week 1 and stay on it).

Track	This experiment's step
A Vitals Monitor	Create a <code>Patient</code> class holding name + a private list of readings; <code>add_reading(...)</code> va
B Pharmacy Stock	Create a <code>Medicine</code> class with private <code>__stock</code> ; <code>dispense(qty)</code> and <code>restock(qty)</code> v
📅 Appointment Book	Create an <code>Appointment</code> class; its setter rejects slots outside clinic hours.
🏖️ Camp Analyzer	Create a <code>Visitor</code> class with validated height/weight and a <code>bmi()</code> method.

Next experiment: specialise your class with a subclass that behaves differently.

✓ Experiment 4 — Inheritance & Polymorphism

Aim

To reuse and extend behaviour across classes using **inheritance**, override methods to specialise behaviour, and use **polymorphism** so the same method call works differently for different hospital-staff objects.

GUIDANCE — concept & hints

Concepts for this experiment

Inheritance lets a new class (the *child / subclass*) reuse the attributes and methods of an existing class (the *parent / base class*): `class Nurse(Staff):` means every `Nurse` automatically gets everything `Staff` has, without rewriting it.

Method overriding happens when a child class defines a method with the *same name* as one in the parent — the child's own version is used for objects of that child class instead of the parent's.

`super()` gives access to the parent class's own version of a method (its `__init__` or any other method) from inside the child, so the child can *reuse and extend* it instead of duplicating its code.

Polymorphism ("many forms") means the same method call — `member.duty()` — runs a different implementation depending on the actual class of the object. This lets you loop over a mixed list of objects and treat them uniformly, while each still behaves in its own way.

✓ Program 1 — Basic Inheritance

Aim of program: Define a base `Staff` class with `name`, `dept` and a `show()` method, then create a `Nurse` subclass that inherits everything without rewriting it.

Algorithm

1. Define `Staff` with `__init__(self, name, dept)` and `show()`.
2. Define `Nurse(Staff)` with no extra code (it inherits everything).
3. Create a `Nurse` object and call `show()`.

GUIDANCE — concept & hints

-  Fill the parent class named in the subclass, and the constructor call.

`class Nurse(Staff):` inherits from `Staff`; calling `Nurse(...)` runs the inherited `Staff.__init__`.

Expected output

Anita - ICU

```
# Program 1 : basic inheritance
class Staff:
    def __init__(self, name, dept):
        self.name = name
        self.dept = dept

    def show(self):
        print(self.name, "-", self.dept)

class Nurse(Staff):
    pass                                     # inherit from Staff

n = Nurse("Anita", "ICU")                  # create a Nurse object (uses Staff.__init__)
n.show()
```

✓ Program 2 — Overriding a Method

Aim of program: Give `Doctor` and `Nurse` their own version of a `role()` method defined in the base `Staff` class, so the same method name behaves differently per class.

Algorithm

1. Define `Staff` with a `role()` method returning a generic description.
2. Define `Doctor(Staff)` overriding `role()` with its own description.
3. Define `Nurse(Staff)` overriding `role()` with its own description.
4. Create one of each and print `role()` for both.

GUIDANCE — concept & hints

- Fill the overriding method's name and the call on the Nurse object.

Re-using the exact name `role` in the subclass overrides the parent's version; call it the same way as any method: `n.role()`.

Expected output

Doctor - diagnoses & treats patients
Nurse - monitors & assists patients

```
# Program 2 : method overriding
class Staff:
    def role(self):
        return "General staff member"

class Doctor(Staff):
    def role(self):
        return "Doctor - diagnoses & treats patients"

class Nurse(Staff):
    def ____ (self):
        # override the same method name
        return "Nurse - monitors & assists patients"

d = Doctor()
n = Nurse()
print(d.role())
print(n.____())
# call the overridden method on
```

✓ Viva-Voce Questions

1. What does a subclass automatically receive from its parent class? (K1 — *Remember*)
2. `Nurse` has no `__init__` of its own — which `__init__` runs for `Nurse("Anita", "ICU")`, and why? (K2 — *Understand*)
3. Add a `Technician` class with its own `role()` — why does the printing loop need **no** change? Name the concept. (K3 — *Apply*)

Result

Thus inheritance, method overriding and polymorphism were implemented in Python using hospital-staff classes, and the outputs were verified.

GUIDANCE — grow your mini-project

Apply what you just practised to **your chosen track** (tracks are described before Experiment 1 — pick one there in week 1 and stay on it).

Track	This experiment's step
 A Vitals Monitor	Add <code>ICUPatient(Patient)</code> that overrides the alert thresholds (stricter limits).
 B Pharmacy Stock	Add <code>Antibiotic(Medicine)</code> that overrides dispensing (e.g. requires a prescription flag).
 C Appointment Book	Add <code>EmergencyAppointment(Appointment)</code> that overrides priority so it jumps the queue

■ Camp Analyzer Add `ChildVisitor(Visitor)` that overrides `bmi_band()` with child-specific cutoffs.

Next experiment: make your data SURVIVE restarts by saving it to a file.

✓ Experiment 5 — Reading & Writing Text Files

CO: CO3 · Sessions: 9–10

Aim

To create, write to and read from text files using Python's file-handling built-ins, keeping simple patient records on disk.

GUIDANCE — concept & hints

Concepts for this experiment

`open(filename, mode)` connects Python to a file on disk. The **mode** decides what you can do: `"w"` creates the file (or *erases* it if it exists) for writing; `"r"` opens it for reading; `"a"` opens it for **appending** — writing new content after whatever is already there, without erasing it.

Using `with open(...) as f:` is the safe pattern — it automatically closes the file for you when the block ends, even if an error occurs.

To write: `f.write(text)` writes exactly the string given (it does *not* add a newline automatically — you add `"\n"` yourself). To read: `f.read()` returns the *whole file* as one string, `f.readlines()` returns a *list* of lines, and looping directly with `for line in f:` reads one line at a time.

`line.strip()` removes the trailing newline (and any extra whitespace) so printed lines don't show a blank line underneath.

✓ Program 1 — Writing Patient Records to a File

Aim of program: Write three patient records to `patients.txt`, one per line, using write mode.

Algorithm

1. Store the records in a list of strings.
2. Open `patients.txt` in write mode.

3. Loop over the list, writing each record followed by a newline.
4. Confirm with a message.

GUIDANCE — concept & hints

Fill the file mode and the write method.

"w" opens the file for writing (creating/overwriting it); `f.write(text)` writes one string.

Expected output

Records written.

```
# Program 1 : writing patient records
records = ["Ravi - Fever", "Meena - Fracture", "Sara - Asthma"]

with open("patients.txt", ____ ) as f:           # mode that creates/overwri
    for r in records:
        f.____(r + "\n")                        # method that writes one li

print("Records written.")
```

✓ Program 2 — Reading Patient Records Back

Aim of program: Open `patients.txt` in read mode, read every line, and print each one without the trailing newline.

Algorithm

1. Open `patients.txt` in read mode.
2. Read all lines into a list with `readlines()`.
3. Loop over the list, printing each line stripped of its newline.

GUIDANCE — concept & hints

Fill the read mode and the method that returns a list of lines.

"r" opens for reading; `f.readlines()` returns every line as a list of strings.

Expected output

Ravi - Fever
Meena - Fracture

```
# Program 2 : reading the records back
with open("patients.txt", "r") as f:
    lines = f.readlines()

for line in lines:
    print(line.strip())
```

mode for reading
method that returns a list

✓ Viva-Voce Questions

1. Which file mode creates/overwrites, which appends, and which reads? (*K1 — Remember*)
2. Why is `with open(...)` safer than calling `open()` and `close()` yourself — what happens if the code in between crashes? (*K2 — Understand*)
3. For a 2 GB hospital log, would you use `f.read()` or `for line in f:`? Justify in terms of memory. (*K4 — Analyse*)

Result

Thus text files were created, written to and read from in Python using patient records, and the outputs were verified.

GUIDANCE — grow your mini-project

Apply what you just practised to **your chosen track** (tracks are described before Experiment 1 — pick one there in week 1 and stay on it).

Track	This experiment's step
 Vitals Monitor	Append every new reading to <code>vitals_log.txt</code> (one line per reading); read the file back t
 Pharmacy Stock	Write closing stock to <code>stock.txt</code> at exit; read it back at start-up.
 Appointment Book	Save the day's bookings to <code>bookings.txt</code> ; reload and print them on restart.
 Camp Analyzer	Append each screening record to <code>camp.txt</code> ; read back the full register.

Next experiment: upgrade your plain-text file to a proper CSV with named columns.

✓ Experiment 6 — File Operations & CSV Records

CO: CO3 · Sessions: 11–12

Aim

To **append** to an existing file without erasing it, and to store and read tabular patient data with Python's `csv` module.

GUIDANCE — concept & hints

Concepts for this experiment

`open(filename, mode)` gave us `"w"` (create/overwrite) and `"r"` (read) last experiment; the third mode is `"a"` — **append** — writing new content after whatever is already there, without erasing it. Text files can't be edited "one line in place": to change or remove a record, programs use the **read-modify-write** pattern — read all lines into a list, change the list, write it all back (exposure only here; you'll meet it again in the mini-project).

A **CSV** (comma-separated values) file stores tabular data — one row per line, fields separated by commas — and Python's built-in `csv` module handles the reading/writing details for you.

`csv.writer(f)` creates a writer whose `.writerow(list)` writes one row from a Python list. `csv.reader(f)` yields each row as a **list of strings**; since the first row is usually a header, `next(reader)` is used once to skip it. `csv.DictReader(f)` is more convenient for named columns: it yields each row as a **dict**, so you read `row["temperature"]` by name. Values from a CSV are always read as **text**, so numeric fields need `int(...)` or `float(...)` before arithmetic.

Always open CSV files with `newline=""` when writing — it prevents blank lines between rows.

✓ Program 1 — Appending a New Record

Aim of program: Add a new patient to `patients.txt` **without** erasing the existing records, then read the whole file back to confirm.

Algorithm

1. (Given) Recreate `patients.txt` with three records, so the experiment is self-contained.
2. Open `patients.txt` in append mode and write one new record.
3. Re-open the file in read mode, read the entire file as one string, and print it.

GUIDANCE — concept & hints

- Fill the append mode and the method that reads the whole file as one string.

"a" opens the file for appending (existing content stays); `f.read()` returns the entire contents as a single string.

Expected output

```
Ravi - Fever
Meena - Fracture
Sara - Asthma
John - Migraine
```

```
# Program 1 : appending a new record
# (setup - given) recreate the records file, so this experiment also
# runs on its own in a fresh Colab session (files do not survive restarts)
with open("patients.txt", "w") as f:
    f.write("Ravi - Fever\n")
    f.write("Meena - Fracture\n")
    f.write("Sara - Asthma\n")

with open("patients.txt", ____ ) as f:                # mode that adds to the end
    f.write("John - Migraine\n")

with open("patients.txt", "r") as f:
    content = f.____()                                # method that reads the wh

print(content)
```

✓ Program 2 — Reading CSV Rows

Aim of program: Read `patients.csv` with `csv.reader`, skip the header, and print each patient's name and age.

Algorithm

1. (Given) Write `patients.csv` (a header + four patient rows) with `csv.writer`, so the experiment is self-contained.
2. Open `patients.csv` and create a `csv.reader` for it.
3. Skip the header row with `next()`.
4. Loop over the remaining rows, printing name and age.

■ GUIDANCE — concept & hints

- Fill the reader constructor and the function that skips the header row.

The (*given*) setup block shows `csv.writer` in action — read it before filling the blanks. `csv.reader(f)` builds the reader; `next(reader)` consumes one row (here, the header) before the loop starts.

Expected output

```
Ravi - 45 yrs
Meena - 30 yrs
Sara - 25 yrs
John - 50 yrs
```

```
# Program 2 : reading rows from a CSV file
import csv

# (setup - given) create patients.csv, so this program is self-contained
data = [
    ["name", "age", "temperature"],
    ["Ravi", 45, 98.6],
    ["Meena", 30, 101.2],
    ["Sara", 25, 99.5],
    ["John", 50, 97.9],
]

with open("patients.csv", "w", newline="") as f:
    writer = csv.writer(f)
    for row in data:
        writer.writerow(row)

with open("patients.csv", "r") as f:
    reader = csv.reader(f)
    header = next(reader)
    for row in reader:
        print(row[0], "-", row[1], "yrs")
```

```
# create a reader object for
# skip/consume the header row
```

✓ Viva-Voce Questions

1. State the three steps of the read–modify–write pattern. (*K1 — Remember*)
2. Every value from `csv.reader` arrives as a string — why, and what must you do before comparing temperatures? (*K2 — Understand*)
3. A new column `ward` is added to `patients.csv` — list every line of your CSV-reading program that must change. (*K3 — Apply*)

Result

Thus file append operations and CSV record processing were implemented in Python, and the outputs were verified.

GUIDANCE — 🌱 grow your mini-project

Apply what you just practised to **your chosen track** (tracks are described before Experiment 1 — pick one there in week 1 and stay on it).

Track	This experiment's step
A Vitals Monitor	Switch the log to <code>vitals.csv</code> with a header (<code>(name,temp,pulse,bp)</code>); use <code>DictReader</code>
B Pharmacy Stock	Keep the register as <code>stock.csv</code> (<code>(medicine,qty,reorder_level)</code>); read it to report low-
C Appointment Book	Store the schedule as <code>schedule.csv</code> (<code>(slot,patient,doctor)</code>); read it to print a day view
D Camp Analyzer	Store the register as <code>camp.csv</code> (<code>(name,age,bmi,band)</code>); read it to count visitors per band

Next experiment: stop crashes — guard every user input with try/except.

✓ Experiment 7 — Exception Handling: try / except

CO: CO4 · Sessions: 13–14

Aim

To handle runtime errors gracefully using `try` / `except`, catching specific exception types instead of letting the program crash.

GUIDANCE — concept & hints

Concepts for this experiment

Code inside a `try:` block is "risky" — if a line raises an error, Python immediately jumps to a matching `except:` block instead of crashing the whole program.

Different mistakes raise different **exception types**: converting bad text with `float("abc")` raises `ValueError`; dividing by zero raises `ZeroDivisionError`; looking up a missing dictionary key raises `KeyError`; indexing past the end of a list raises `IndexError`.

You can stack **multiple** `except` blocks under one `try` — Python checks them top to bottom and runs the *first* one that matches the error that occurred. If two different errors should be handled the *same way*, catch both in **one** block using a tuple:

```
except (KeyError, IndexError):
```

A generic `except Exception as e:` is a catch-all for anything you didn't specifically anticipate — `e` holds the actual exception object, and printing it shows its error message.

✓ Program 1 — Safe Temperature Input

Aim of program: Read a temperature from the user and print it, catching the error if the text can't be converted to a number.

Algorithm

1. Inside `try`, read input and convert it to `float`.
2. Print the recorded temperature.
3. In `except`, catch the conversion error and print a friendly message.

GUIDANCE — concept & hints

 Fill the conversion function and the exception type it can raise.

`float(...)` converts text to a decimal; a failed conversion raises `ValueError`.

Expected output

```
Enter temperature: abc
Invalid input: please enter a number.
```

```
# Program 1 : safe temperature input
try:
    temp = ____ (input("Enter temperature: "))    # convert text to float (ma
    print("Temperature recorded:", temp)
except ____:                                     # exception raised when th
    print("Invalid input: please enter a number.")
```

✓ Program 2 — BMI with Two Exception Types

Aim of program: Compute BMI from a weight and height entered by the user, handling **bad text** and a **zero height** with two separate, specific `except` blocks.

Algorithm

1. Inside `try`, read weight and height as floats and compute BMI.
2. Print the BMI.
3. First `except`: catch bad numeric text.
4. Second `except`: catch a height of zero.

GUIDANCE — concept & hints

● Fill the two exception types, in the order they should be checked.

Bad text raises `ValueError`; dividing by a height of `0` raises `ZeroDivisionError`.

Expected output

```
Enter weight in kg: 60
Enter height in m: 0
Height cannot be zero.
```

```
# Program 2 : BMI with two exception types
try:
    weight = float(input("Enter weight in kg: "))
    height = float(input("Enter height in m: "))
    bmi = weight / (height ** 2)
    print("BMI =", round(bmi, 2))
except ____:                # raised when the text ca
    print("Invalid input: please enter numbers.")
except ____:                # raised when height is z
    print("Height cannot be zero.")
```

✓ **Viva-Voce Questions**

1. Name the exception raised by `float("abc")` and the one raised by `10 / 0`.
(K1 — Remember)
2. The moment an exception is raised, what happens to the remaining lines of the `try` block? (K2 — Understand)
3. Wrap the temperature input in a loop that keeps asking until the user enters a valid number — which statement ends the loop? (K3 — Apply)

Result

Thus runtime errors were handled gracefully in Python using `try/except` with specific exception types, and the outputs were verified.

 GUIDANCE — **grow your mini-project**

Apply what you just practised to **your chosen track** (tracks are described before Experiment 1 — pick one there in week 1 and stay on it).

Track	This experiment's step
 A Vitals Monitor	Wrap vitals entry in <code>try/except</code> so bad text or a zero height prints a friendly message and
 B Pharmacy Stock	Guard the quantity input, and handle a lookup of a medicine that doesn't exist (<code>KeyError</code>).
 C Appointment Book	Handle a non-numeric or out-of-range slot number with two specific <code>except</code> blocks.

Next experiment: create your project's OWN exception type and raise it yourself.

Experiment 8 — raise, finally & User-defined Exceptions

CO: CO4 · Sessions: 15–16

Aim

To manually trigger exceptions with `raise`, guarantee cleanup with `finally`, and define **custom, domain-specific exception classes** that are raised and caught like built-ins.

GUIDANCE — concept & hints

Concepts for this experiment

`raise` lets *your own code* trigger an exception on purpose, not just when Python itself hits an error — `raise ValueError("message")` stops execution right there and hands that message to whichever `except` catches it.

`finally:` is a block that runs **no matter what** — whether the `try` succeeded or failed. It's the right place for cleanup steps that must never be skipped.

Every built-in exception belongs to a **hierarchy**: `ValueError`, `ZeroDivisionError` and the rest are all subclasses of `Exception`, so `except Exception:` catches any of them — list specific handlers *before* a general one.

A **custom exception** is just a class that inherits from `Exception`: `class InvalidAgeError(Exception): pass` creates a brand-new, meaningfully-named error you can `raise` and `except` exactly like a built-in. *Exposure only*: custom exceptions can carry their own data (via `__init__` + `super().__init__(message)`) and can form their own small hierarchies under one custom base class.

Program 1 — Manually Raising a ValueError

Aim of program: Write a function that rejects a negative age by *raising* a

`ValueError` with a custom message, and catch it to print that message.

Algorithm

1. Define `validate_age(age)`: if `age < 0`, raise `ValueError` with a message; else return it.
2. Call the function inside `try` with a negative age.
3. Catch the exception and print its message.

GUIDANCE — concept & hints

 **Fill the keyword that triggers the exception, and the name that holds the caught exception.**

`raise ValueError("...")` triggers the exception; the exception object caught as `e` prints as its message.

Expected output

```
Error: Age cannot be negative
```

```
# Program 1 : manually raising a ValueError
def validate_age(age):
    if age < 0:
        ____ ValueError("Age cannot be negative")    # manually trigger this
    return age

try:
    print("Valid age:", validate_age(-5))
except ValueError as e:
    print("Error:", ____)                                # the exception's messa
```

✓ Program 2 — A Simple Custom Exception

Aim of program: Define `InvalidAgeError`, raise it for a negative age, and catch it like any other exception.

Algorithm

1. Define `InvalidAgeError` inheriting from `Exception`.
2. Define `check_age(age)`: raise `InvalidAgeError` if `age < 0`.
3. Call it inside `try/except`, catching the custom exception and printing its message.

GUIDANCE — concept & hints

 Fill the base class to inherit from, and the exception type to catch.

Every custom exception inherits from `Exception`; catch it by name, just like a built-in: `except InvalidAgeError`.

Expected output

```
Error: Age cannot be negative
```

```
# Program 2 : a simple custom exception
class InvalidAgeError(____):                # base class every custom e
    pass

def check_age(age):
    if age < 0:
        raise InvalidAgeError("Age cannot be negative")
    return age

try:
    check_age(-5)
except ____ as e:                          # catch our custom excepti
    print("Error:", e)
```

✓ Viva-Voce Questions

1. Which keyword manually triggers an exception, and which block runs no matter what? (*K1 — Remember*)
2. Why is `except ValueError:` better practice than a bare `except:`? (*K2 — Understand*)
3. Write `validate_bp(systolic)` that raises `ValueError` outside 70–250 and returns the value otherwise. (*K3 — Apply*)

Result

Thus exceptions were manually raised, cleanup guaranteed with `finally`, and custom exception classes defined, raised and caught, and the outputs were verified.

GUIDANCE — grow your mini-project

Apply what you just practised to **your chosen track** (tracks are described before Experiment 1 — pick one there in week 1 and stay on it).

Track	This experiment's step
 Vitals Monitor	Define <code>InvalidReadingError(Exception)</code> ; <code>add_reading</code> raises it for impossible value;
 Pharmacy Stock	Define <code>LowStockError(Exception)</code> ; <code>dispense</code> raises it when the request exceeds stock;
 Appointment Book	Define <code>DoubleBookingError(Exception)</code> ; booking a taken slot raises it.
 Camp Analyzer	Define <code>InvalidMeasurementError(Exception)</code> ; impossible height/weight raises it during measurement.

Next experiment: analyse everything you've collected with pandas.

✓ Experiment 9 — Data Analysis with Pandas

CO: CO5 · **Sessions:** 17–18

Aim

To build, load and filter tabular patient data using the **pandas** library's **DataFrame**.

GUIDANCE — concept & hints

Concepts for this experiment

A pandas **DataFrame** is a table: rows and named columns, like a spreadsheet inside Python. You can build one directly from a dictionary of equal-length lists with

```
pd.DataFrame({...}), or load one straight from a file with
pd.read_csv("file.csv").
```

Access a single column with `df["column_name"]` — this returns a **Series** (one labeled column of values). `df.head(n)` previews the first `n` rows without printing the whole table; `df.shape` gives `(rows, columns)`.

Boolean indexing filters rows: `df[df["temperature"] >= 100.4]` keeps only the rows where that condition is `True`.

`df.groupby("column")` groups rows that share the same value in that column; following it with `["other_column"].mean()` (or `.sum()`, `.count()`, ...) computes that statistic **per group**.

✓ Program 1 — Building a DataFrame

Aim of program: Build a DataFrame directly from a dictionary of patient vitals and inspect one of its columns.

Algorithm

1. Import pandas as `pd`.
2. Store patient data as a dict of equal-length lists.
3. Build a DataFrame from the dict.
4. Print the whole DataFrame, then just the `age` column.

GUIDANCE — concept & hints

Fill the DataFrame constructor and the column name.

`pd.DataFrame(data)` builds a table from the dict; `df["age"]` selects one column.

Expected output

```
   name  age  temperature
0  Ravi   45         98.6
1  Meena  30        101.2
2   Sara  25         99.5
3   John  50         97.9
0     45
1     30
2     25
3     50
Name: age, dtype: int64
```

```
# Program 1 : building a DataFrame
import pandas as pd

data = {
    "name": ["Ravi", "Meena", "Sara", "John"],
    "age": [45, 30, 25, 50],
    "temperature": [98.6, 101.2, 99.5, 97.9],
}

df = pd.__(data)           # build a DataFrame from t
print(df)
print(df["__"])           # print just the age column
```

✓ Program 2 — Loading a CSV and Filtering Rows

Aim of program: Write patient vitals to `vitals.csv`, load it into a DataFrame with `read_csv` and preview it, then keep only the patients with a fever (`temperature >= 100.4`) using boolean indexing.

Algorithm

1. (Given) Write a small CSV file with a header and four patient rows.
2. Load it into a DataFrame using `pd.read_csv`; print the first 2 rows and the shape.
3. Build a boolean mask comparing the `temperature` column to `100.4` and filter with it.
4. Print the matching names and temperatures, and how many rows matched.

GUIDANCE — concept & hints

● Fill the CSV-loading function, the preview method, and the column compared to the threshold.

`pd.read_csv("vitals.csv")` reads the file straight into a DataFrame;
`df.head(2)` shows only the first 2 rows; `df["temperature"] >= 100.4` builds the boolean mask that keeps only the matching rows.

Expected output

```
   name  age  temperature
0  Ravi   45         98.6
1  Meena  30        101.2
Shape: (4, 3)
   name  temperature
1  Meena        101.2
Fever count: 1
```

```
# Program 2 : loading a CSV into a DataFrame and filtering rows
import pandas as pd

# (setup - given) write the CSV file, so this program is self-contained
with open("vitals.csv", "w") as f:
    f.write("name,age,temperature\n")
    f.write("Ravi,45,98.6\n")
    f.write("Meena,30,101.2\n")
    f.write("Sara,25,99.5\n")
    f.write("John,50,97.9\n")

df = pd.____("vitals.csv")          # read the CSV file into a
print(df.____(2))                  # show only the first 2 ro
print("Shape:", df.shape)

fever_df = df[df[["____"]] >= 100.4]  # column to compare against
print(fever_df[["name", "temperature"]])
print("Fever count:", len(fever_df))
```

✓ Viva-Voce Questions

1. What is a DataFrame, and what is a single column of it called? (*K1 — Remember*)
2. What does `df["temperature"] >= 100.4` produce on its own, *before* it is used for filtering? (*K2 — Understand*)
3. Write the one-liner that shows only `name` and `age` of patients older than 40. (*K3 — Apply*)

Result

Thus patient vitals data was built, loaded, filtered and summarised using pandas DataFrames, and the outputs were verified.

GUIDANCE — grow your mini-project

Apply what you just practised to **your chosen track** (tracks are described before Experiment 1 — pick one there in week 1 and stay on it).

Track	This experiment's step
 Vitals Monitor	Load <code>vitals.csv</code> with pandas; per-patient mean/min/max temperature; filter out-of-range.
 Pharmacy Stock	Load <code>stock.csv</code> ; filter items below reorder level; add a total-stock summary.
 Appointment Book	Load <code>schedule.csv</code> ; count appointments per doctor (<code>groupby</code>).
 Camp Analyzer	Load <code>camp.csv</code> ; count visitors per band and mean BMI per age group (<code>groupby</code>).

Final experiment next: one chart turns your project into a dashboard.

✓ Experiment 10 — Visualization: Matplotlib & Seaborn

CO: CO5 · Sessions: 19–20

Aim

To visualize patient data with **Matplotlib** (a labelled line chart) and **Seaborn** (a distribution histogram), and to see an **interactive Plotly chart** in action.

GUIDANCE — concept & hints

Concepts for this experiment

Matplotlib is Python's standard charting library; its `pyplot` module is imported as `plt`. Build a chart with one drawing call — `plt.plot(x, y)` for a **line**, `plt.bar(labels, values)` for **bars**, `plt.scatter(x, y)` for **points** — then dress

it with `plt.xlabel()`, `plt.ylabel()`, `plt.title()`, and finally `plt.show()`. In Colab the chart appears inline, right below the cell. Choose the chart for the question: a **line** shows change over time, **bars** compare categories, a **scatter** reveals a relationship.

Seaborn (`import seaborn as sns`) sits on top of Matplotlib and speaks

DataFrame: `sns.histplot(data=df, x="systolic")` handles binning, colours and styling. A **histogram** shows how readings are *distributed* — how many fall in each range. You still finish with `plt.title(...)` and `plt.show()`.

Exposure only — try it in a scratch cell: **Plotly Express** makes charts you can **touch** (hover, zoom, pan). Paste and run:

```
import plotly.express as px
import pandas as pd
df = pd.DataFrame({
    "name":      ["Ravi", "Meena", "Sara", "John", "Devi", "Arun"],
    "age":       [45, 30, 25, 50, 62, 38],
    "systolic":  [130, 121, 115, 138, 149, 124],
    "ward":      ["ICU", "Ward-A", "Ward-A", "ICU", "ICU", "Ward-B"],
})
fig = px.scatter(df, x="age", y="systolic", color="ward",
                 hover_name="name", title="Age vs systolic BP by ward")
fig.show()
```

✓ Program 1 — Line Chart — Temperature Through the Day

Aim of program: Plot one patient's six hourly temperature readings as a line chart and label it fully.

Algorithm

1. Store hours 1–6 and the six temperature readings in two lists.
2. Draw a line chart of temperature against hour, with point markers.
3. Label both axes and add a title and grid.
4. Display the chart, then print how many readings were plotted.

GUIDANCE — concept & hints

 **Fill the drawing / labelling / display calls** — one word each.

The line-drawer is `plot`, the x-axis label call is `xlabel`, and nothing appears until `show()`. A line chart appears below the cell.

Expected output

Line chart drawn with 6 readings

```
# Program 1 : line chart - temperature through the day
import matplotlib.pyplot as plt

hours = [1, 2, 3, 4, 5, 6]           # observation hour
temps = [98.6, 99.1, 100.2, 101.0, 99.8, 98.9] # deg F at each hour

plt.__(hours, temps, marker="o")      # draw a LINE chart with dot
plt.__( "Hour of observation")        # label the x-axis
plt.ylabel("Temperature (F)")
plt.title("Patient temperature - Day 1")
plt.grid(True)
plt.__()                             # display the chart

print("Line chart drawn with", len(temps), "readings")
```

✓ Program 2 — Seaborn Histogram — BP Distribution

Aim of program: Show how ten systolic BP readings are distributed using a Seaborn histogram.

Algorithm

1. Build a one-column DataFrame of ten systolic readings.
2. Draw a histogram of that column with 5 bins.
3. Add a title, display the chart, and print how many readings were plotted.

GUIDANCE — concept & hints

Fill the Seaborn function and the column name.

Seaborn's histogram function is `histplot`; the column is named in `x="..."`. A histogram appears below the cell.

Expected output

Readings plotted: 10

```
# Program 2 : Seaborn histogram - distribution of systolic BP
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

```
df = pd.DataFrame({
    "systolic": [112, 118, 121, 130, 138, 145, 151, 126, 133, 120],
})

sns.____(data=df, x="____", bins=5)           # histogram of the systolic
plt.title("Distribution of systolic BP")
plt.show()

print("Readings plotted:", len(df))
```

✓ Viva-Voce Questions

1. Which `pyplot` functions draw a line chart, a bar chart and a scatter plot? (*K1 — Remember*)
2. What does a histogram tell you about BP readings that a line chart of the same numbers cannot? (*K2 — Understand*)
3. Add Day 2's readings as a second line on Program 1's chart with a legend — which two calls change or get added? (*K3 — Apply*)

Result

Thus patient data was visualized with a Matplotlib line chart and a Seaborn histogram, and the outputs were verified.

GUIDANCE — grow your mini-project

Apply what you just practised to **your chosen track** (tracks are described before Experiment 1 — pick one there in week 1 and stay on it).

Track	This experiment's step
 A Vitals Monitor	Line chart of one patient's temperature trend + histogram of all readings.
 B Pharmacy Stock	Bar chart of current stock per medicine (low-stock bars stand out).
 C Appointment Book	Bar chart of appointments per doctor or per slot.
 D Camp Analyzer	Histogram of BMI values + bar chart of visitors per band.

That's every piece built. Assemble them into one program in the Mini-Project section below — menu loop + class + CSV + custom exception + chart.

✓ Mini-Project — Final Assembly

CO: CO1–CO5 · **Sessions:** 21–28 · **Mode:** individual or pairs (as announced)

Aim

To assemble the pieces you built after each experiment — on **your chosen track** — into one small but complete Python application: a menu loop, your class (and subclass), CSV persistence, your custom exception, and one chart.

GUIDANCE — final-assembly checklist

By now every part of your project already exists in your 🌱 steps. Assembly means: put them in one program, in this order — 1) imports · 2) your exception class · 3) your class & subclass · 4) file/CSV load-save functions · 5) analysis/chart functions · 6) the menu loop that ties them together.

Must-haves (all tracks): a menu loop that runs until *quit* · at least one class with a subclass · records persisted to CSV and reloaded on start · your custom exception raised **and** caught · one pandas summary · one chart.

Assessment: correctness 40% · code quality & comments 20% · breadth of course concepts used 20% · demo & viva 20%.

✓ My Mini-Project

Field	Details
Track	
Student(s)	
COs covered	

Aim

(one or two sentences: what your application does)

Design

(the functions / classes you built, the data files used, and how they connect — a short list is enough)

Program

(write your full program in the code cell below; add more cells if needed)

```
# ===== MINI-PROJECT CODE =====  
# Write your full program here. Keep it organised:  
# 1) imports                2) data / constants  
# 3) classes & functions 4) main program / menu loop  
# Comment every section - comments are part of the marks!
```

Output